Classical ILC/DRP Design from an Algebraic (Matrix-Fraction) Perspective – Part 1: Framework

Kevin L. Moore

Department of Electrical Engineering
Colorado School of Mines
Golden, Colorado, USA

Pre-Congress Worskhop WS-13 – Learning from Data: Principles, Methods, and Emerging Ideas & Applications in Iterative Learning Control

2026 IFAC World Congress
Busan, Republic of Korea
23 August 2026

Dedicated to the Memory of YangQuan Chen

A Simple Idea ...

...old enough to be one of those Latin phrases that's engraved into stone: *Repetitio mater studiorum est*, or **"Repetition is the mother of all learning"**

www.brainscape.com/blog/2011/05/repetition-is-the-mother-of-all-learning

Control Paradigms Involving Repetition

- **Iterative Learning Control** (ILC)
  - For systems that seek to execute the same trajectory over-and-over from the same starting point each time
  - In this case the control input is adapted from one trial to the next

Control Paradigms Involving Repetition

- **Iterative Learning Control** (ILC)
  - For systems that seek to execute the same trajectory over-and-over from the same starting point each time
  - In this case the control input is adapted from one trial to the next
- **Repetitive Control** (RC)
  - For systems subject to periodic references or disturbances
  - For systems with periodic parameter variations

Control Paradigms Involving Repetition

- **Iterative Learning Control** (ILC)
  - For systems that seek to execute the same trajectory over-and-over from the same starting point each time
  - In this case the control input is adapted from one trial to the next
- **Repetitive Control** (RC)
  - For systems subject to periodic references or disturbances
  - For systems with periodic parameter variations
- **(Discrete) Repetitive Processes** (DRP)
  - Same operational scenario as ILC, but ...
  - System has inherent trial-to-trial memory

In This Two-Part Talk ...

- **Part 0: Brief Overview of two of these paradigms:**
  - **ILC**
  - **DRP**
- Part 1: Analysis and design framework for such systems
  - Lifted models
    - The "supervector" framework of ILC
    - The update law using supervector notation
    - The ILC design problem
    - A lifting approach to DRPs
  - The w-transform and operator perspective
  - Generalization and closed-loop control of DRPs
  - Complete framework
- Part 2: Design for discrete ILC/DRP systems

Iterative Learning Control (ILC)

- **Iterative Learning Control**: Paradigm for processes or systems that operate in an iterative or repetitive fashion:
  - Finite time or spatial interval of operation
  - Return to some starting point or state between repetitions
  - Improve the performance from one operation to the next.

Iterative Learning Control (ILC)

- **Iterative Learning Control**: Paradigm for processes or systems that operate in an iterative or repetitive fashion:
  - Finite time or spatial interval of operation
  - Return to some starting point or state between repetitions
  - Improve the performance from one operation to the next.

- Such systems are known as
  - Iterative
  - Trial-to-trial
  - Multi-pass
  - Repetitive
  - Periodic
  - Batch-to-batch
  - Run-to-run
  - Pass-to-pass
  - ...

Iterative Learning Control (ILC)

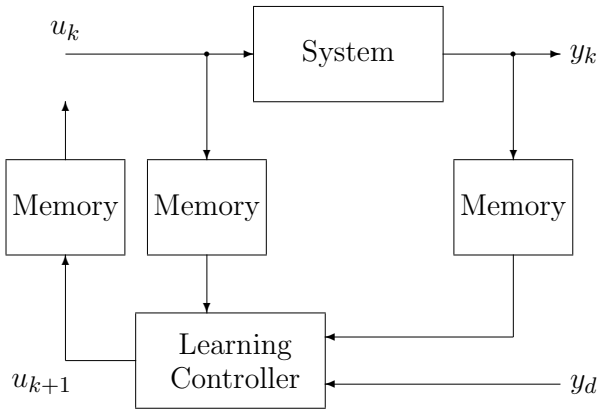
- **Iterative Learning Control**: Paradigm for processes or systems that operate in an iterative or repetitive fashion:
  - Finite time or spatial interval of operation
  - Return to some starting point or state between repetitions
  - Improve the performance from one operation to the next.

- Such systems are known as
  - Iterative
  - Trial-to-trial
  - Multi-pass
  - Repetitive
  - Periodic
  - Batch-to-batch
  - Run-to-run
  - Pass-to-pass
  - …

- Such systems arise in a variety of applications, including:
  - Process control
  - Semiconductor processing
  - Robotic motion control
  - Hard disk drive servo control
  - Machining and manufacturing operations
  - …

ILC Algorithms

- Standard iterative learning control scheme:



- **Goal**: Find a learning control algorithm

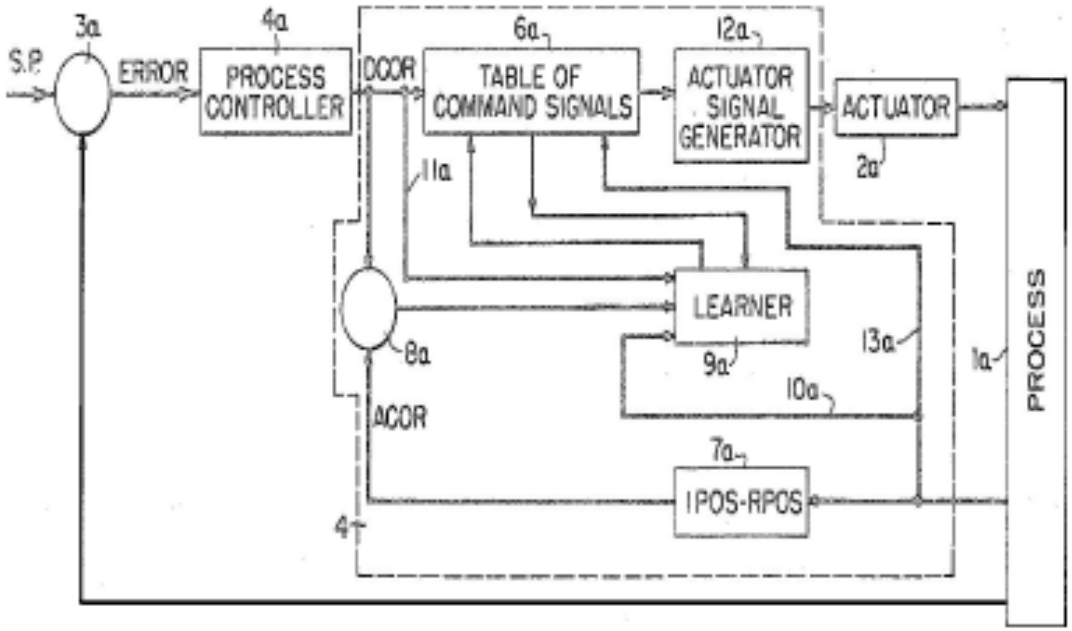
$$u_{k+1}(t) = f_L(\text{previous information})$$

so that for all $t \in [0, t_f]$

$$\lim_{k\to\infty} y_k(t) = y_d(t)$$

On the History and Accomplishments of ILC

- United States Patent 3,555,252 – Learning Control of Actuators in Control Systems," filed 1967, awarded 1971, learned characteristics of actuators and used this knowledge to correct command signals.



- "Comments on United States Patent 3,555,252 – Learning Control of Actuators in Control Systems," Yangquan Chen and Kevin L. Moore, in *Proceedings of the 2000 International Conference on Automation, Robotics, and Control*, Singapore, December 2000.

First ILC paper- in Japanese (1978)

- M. Uchiyama, "Formulation of high-speed motion pattern of a mechanical arm by trial," *Trans. SICE (Soc. Instrum. Contr. Eng.)*, 14(6):706–712(in Japanese), 1978.

706

試行による人工の手の高速運動パターンの形成†

内山勝*

Formation of High-Speed Motion Pattern of a Mechanical Arm by Trial

Masaru UCHIYAMA*

High-speed motion of a mechanical arm is necessary to speed up a job done by the arm. In high speed, however, the desired trajectory of motion of the arm cannot be obtained simply by applying the trajectory function to the servo system as the reference function because the time lag in the servo system is not negligible.

A solution to this problem is to apply dynamically compensating computed torques to the servo system. By this method, however, for increasing the accuracy of the mathematical model of the arm necessary to compute the compensating torques, a very large effort would be required. To avoid this difficulty, an alternative method of correcting the reference function by trial will be useful. Repeating a proper process of trial and correction, the reference function which realizes the desired pattern of trajectory may be obtained.

In this paper, correcting algorithm of a reference function for this method is investigated theoretically from the standpoint of stability or convergency of the process of trial and correction, and a stable correcting algorithm is obtained. Through the experiment using a mechanical arm of six degrees of freedom controlled by a digital computer, it is confirmed that the process of trial and correction by this algorithm is stable and the response of the servo system converges rapidly to the desired pattern of trajectory.

1. まえがき

人工の手による作業の能率を高めるためには，高速の運動を行わせる必要があるが，高速運動時には，各関節を駆動するサーボ系の動特性により，指令値に対する応答の遅れが無視できなくなる．したがって，目標とする各関節角度の運動軌道の関数をそのままサーボ系への指令値として出力したのでは，精度のよい運動軌道の試現は期待できない．

これに対して，Paul は，人工の手の数学モデルから動的な補償トルクを計算し，出力することにより，精度のよい運動軌道の創成を実現している[1]．しかし，この方法では，モデルの精度を高めると，補償トルクの計算が膨大になる．また，モデルのパラメータの同定もやっかいな問題として残る．

ところで，人工の手の作業においては，産業用ロボットの動作に見られるように，あらかじめ決められた運動パターンの繰返し動作が多く現れる．このような動作では，与えられた運動パターンを実現するためのサーボ系への指令値を試行と修正を繰り返すことにより，あらかじめ形成しておくことが可能である．

本論文では，試行に基づく指令値の修正方法を，試行と修正の過程の安定性という観点から，理論的に検討し，試行の修正アルゴリズムを求める．そして，このアルゴリズムにより，安定な試行と修正の過程が実現できることを 6 自由度の人工の手を使用した実験により確かめる．

2. 修正アルゴリズム

2.1 問題設定

目標とする運動軌道を $\theta_d(t)$ $(0 \leq t \leq T)$ としよう．ここで，$\theta_d(t)$ は，人工の手の i 番目の関節角度を時間 t の関数として表した関数 $\theta_{di}(t)$ $(i=1 \sim n)$ を第 i 成分とする n 次元ベクトルである．n は人工の手の自由度の数である．添字 d は $\theta_d(t)$ が目標軌道であることを表す．T は運動時間である．試行においては，人工の手のサーボ系に適当な位置指令 $\theta_{ri}(t)$ を与え，その応答 $\theta_{ai}(t)$ を測定する．ここで，添字 r は $\theta_r(t)$ が指令値関数であることを，添字 a は $\theta_a(t)$ がサーボ系の応答であることを表す．また，添字 j は j 回目の試行を表す．そして，$\theta_{aj}(t)$ を $\theta_r(t)$ と比較することにより，$\theta_{r,j+1}(t)$ が $\theta_r(t)$ によりいっそう近づくように，つぎの試行の位置指令 $\theta_{r,j+1}(t)$ を決定する．$\theta_r(t)$, $\theta_{rj}(t)$, $\theta_{aj}(t)$ から $\theta_{r,j+1}(t)$ を計算するアルゴリズムを修正アルゴリズムと呼ぶことにする．

† 第 18 回自動制御連合講演会で発表（昭 50・11）
* 東北大学工学部 Faculty of Engineering, Tohoku University, Sendai (Received August 5, 1977)

— 94 —

内山：試行による人工の手の高速運動パターンの形成 707

修正アルゴリズムにおいては，試行における偏差の関数

$$e_j(t)=\theta_r(t)-\theta_{aj}(t)$$

に注目して $\theta_{r,j+1}(t)$ を算出するのが自然な方法であると考えられる．このような方法による修正の一般式は，

$$\theta_{r,j+1}(t)=\theta_{r,j+1}(\theta_{rj}(t), e_j(t))$$

により与えられる．ここでは簡単のために，修正を各自由度ごとに独立して行い，第 i 自由度の修正を

$$\theta_{ri,j+1}=\theta_{rij}+\Psi_{ij}E_{ij} \quad (1)$$

により行うものに限定して，理論的な検討を進めることとする．ただし，大文字は関数のラプラス変換形を表すものとする．

本論文では，問題をこのように限定しても，多くの自由度について

$$\lim_{j\to\infty}\frac{1}{T_0}\int_0^{T_0}\{e_{ij}(t)\}^2dt=0$$

となるような Ψ_{ij} の形を決定する方法を示す．ただし，T_0 は T よりも適当に長い時間とする．なお，当然のことながら，この問題において $\theta_r(t)$ はサーボ系の各アクチュエータの出力パワーの許容値内で実現可能な関数でなければならない．

2.2 試行の遷移関数

試行と修正の過程の安定性とは，各段階の試行での各自由度の 2 乗平均誤差の平方根

$$\varepsilon_{ij}=\sqrt{\frac{1}{T_0}\int_0^{T_0}\{e_{ij}(t)\}^2dt}$$

が，試行を重ねるにつれて減少していく傾向にあることをいうものとする．このような試行と修正の過程の安定性は，

$$E_{i,j+1}=\Phi_{ij}E_{ij} \quad (2)$$

により定義される試行の遷移関数 Φ_{ij} により調べることができる．Φ_{ij} は Ψ_{ij} の関数としてつぎのように求められる．

人工の手の各関節を駆動するサーボ系は，各サーボの間で角速度，角加速度に基づく複雑な干渉がある 2 次の多自由度サーボ系である．これらの干渉を各サーボに加わる外乱とみなし，位置指令と等価な量に換算して ξ とおくと，サーボ系の運動方程式は

$$I_i\ddot{\theta}_{ai}+K_{vi}\dot{\theta}_{ai}+K_{ti}\theta_{ai}=K_{ti}(\theta_{ri}+\xi) \quad (3)$$

となる．ここで，I_{ti}, K_{vi}, K_{ti} は，それぞれ I_{ti}, K_{vi}, K_{ti} を対角成分とする $n\times n$ の対角マトリクスである．

Fig. 1 Simplified model of servomechanism of each articulation

る．(3)式により，各自由度のサーボをそれぞれ独立に取り扱うことが可能となる．ただし，I_{ti} は $\theta_{i+1}, \theta_{i+2}, \cdots, \theta_n$ の関数で

$$I_{ti}=I_{ti}(\theta_{i+1}, \theta_{i+2}, \cdots, \theta_n) \quad (4)$$

と表される．したがって，各関節のサーボには一定した伝達関数は定義できない．しかし，各段階の試行で行われる一定の運動に対して，各サーボの重み関数 $g_{ti}(t)$ を

$$\theta_{ai}(t)=\int_0^t g_{ti}(t-\tau)\{\theta_{ri}(\tau)+\xi_i(\tau)\}d\tau$$

$$\theta_{ri,j+1}=\theta_{ri,j}+\psi_{ij}E_{ij}$$

る．

さて，各回の試行における各自由度の修正量が小さいと仮定しよう．すなわち，

$$\theta_{ri,i+1}\approx\theta_{ri} \quad (5)$$

とする．このように仮定すると，

$$G_{ti,i+1}\simeq G_{ti} \quad (7)$$

$$\Xi_{i,i+1}\simeq\Xi_{ti} \quad (8)$$

がいえる．ただし，$F_i(s)\simeq F_i(s)$ は $f_i(t)-f_i(t)\to 0$ とする．ここで，$f_i(t), f_i'(t)$ はラプラス変換が可能な任意の関数とし，$\|\cdot\|$ は関数のノルムを表すものとする．(1)，(5)，(7)，(8)式より

$$E_{i,j+1}=\Theta_{ri}-\theta_{ai,j+1}$$
$$=(1-G_{ti}\Psi_{ij})E_{ij}$$

となる．これを(2)式と比較して，

$$\Phi_{ij}=1-G_{ti}\Psi_{ij} \quad (9)$$

を得る．

2.3 修正アルゴリズムの条件

修正アルゴリズムは試行と修正の過程が安定になるように決める必要がある．この安定性は遷移関数 Φ_{ij} により調べることができる．ここでは，Φ_{ij} による遷移の安定条件を周波数領域における解析から導こう．

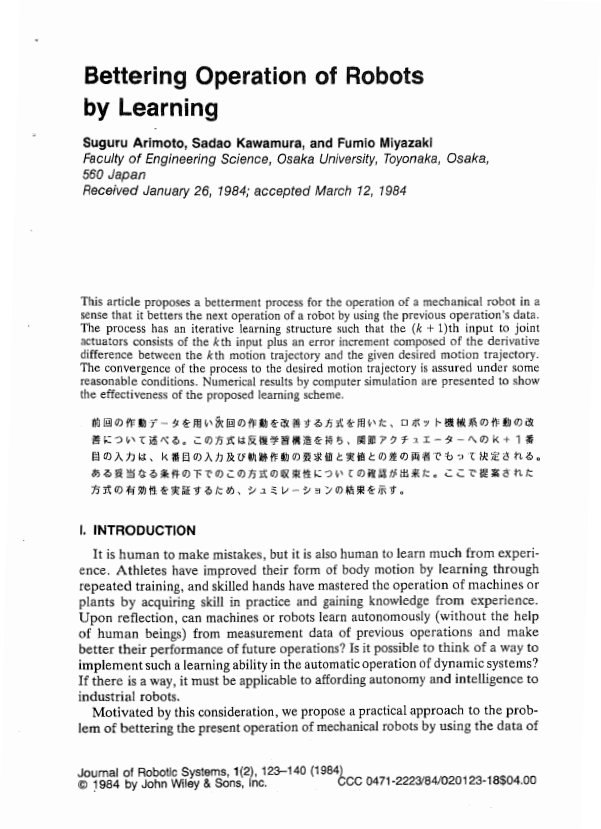
2.1 節で述べたように，T_0 を応答が整定する程度に適当に長い時間であるとしよう．すると，$e_{ij}(t)$ は周期 T_0 の周期関数により近似でき，フーリエ級数

— 95 —

$$\theta_{ri,j+1}=\theta_{ri,j}+\psi_{ij}E_{ij}$$

First ILC paper- in English (1984)

S. Arimoto, S. Kawamura, and F. Miyazaki, "Bettering operation of robots by learning," *J. of Robotic Systems*, 1(2):123–140, 1984.



Bettering Operation of Robots by Learning

Suguru Arimoto, Sadao Kawamura, and Fumio Miyazaki
Faculty of Engineering Science, Osaka University, Toyonaka, Osaka, 560 Japan
Received January 26, 1984; accepted March 12, 1984

This article proposes a betterment process for the operation of a mechanical robot in a sense that it betters the next operation of a robot by using the previous operation's data. The process has an iterative learning structure such that the $(k+1)$th input to joint actuators consists of the kth input plus an error increment composed of the derivative difference between the kth motion trajectory and the given desired motion trajectory. The convergence of the process to the desired motion trajectory is assured under some reasonable conditions. Numerical results by computer simulation are presented to show the effectiveness of the proposed learning scheme.

前回の作動データを用い次回の作動を改善する方式を用いた。ロボット機械系の作動の改善について述べる。この方式は反復学習構造を持ち、関節アクチュエーターへのK＋1番目の入力は、K番目の入力及び軌跡作動の要求値と実績との差の両者でもって決定される。ある妥当なる条件の下でのこの方式の収束性についての確認が出来た。ここで提案された方式の有効性を実証するため、シュミレーションの結果を示す。

I. INTRODUCTION

It is human to make mistakes, but it is also human to learn much from experience. Athletes have improved their form of body motion by learning through repeated training, and skilled hands have mastered the operation of machines or plants by acquiring skill in practice and gaining knowledge from experience. Upon reflection, can machines or robots learn autonomously (without the help of human beings) from measurement data of previous operations and make better their performance of future operations? Is it possible to think of a way to implement such a learning ability in the automatic operation of dynamic systems? If there is a way, it must be applicable to affording autonomy and intelligence to industrial robots.

Motivated by this consideration, we propose a practical approach to the problem of bettering the present operation of mechanical robots by using the data of

Journal of Robotic Systems, 1(2), 123–140 (1984)

CCC 0471-2223/84/020123-18$04.00

Professor Arimoto's Early Contributions

- Professor Arimoto's Six Postulates of ILC:
  - **P1**. Every trial (pass, cycle, batch, iteration, repetition) ends in a fixed time of duration $T > 0$.
  - **P2**. A desired output $y_d(t)$ is given *a priori* over $[0, T]$.
  - **P3**. Repetition of the initial setting is satisfied, that is, the initial state $x_k(0)$ of the objective system can be set the same at the beginning of each iteration: $x_k(0) = x^0$, for $k = 1, 2, \cdots$.
  - **P4**. Invariance of the system dynamics is ensured throughout these repeated iterations.
  - **P5**. Every output $y_k(t)$ can be measured and therefore the tracking error signal, $e_k(t) = y_d(t) - y_k(t)$, can be utilized in the construction of the next input $u_{k+1}(t)$.
  - **P6**. The system dynamics are invertible, that is, for a given desired output $y_d(t)$ with a piecewise continuous derivative, there exists a unique input $u_d(t)$ that drives the system to produce the output $y_d(t)$.
- Professor Arimoto proposed a learning control algorithm of the form:

$$u_{k+1}(t) = u_k(t) + \Gamma \dot{e}_k(t)$$

  Convergence is assured if $\|I - CB\Gamma\|_i < 1$.
- Professor Arimoto also considered more general algorithms of the form:

$$u_{k+1} = u_k + \Phi e_k + \Gamma \dot{e}_k + \Psi \int e_k dt$$

Other Learning Control Algorithms

- Numerous studies have considered a variation on the theme of the basic Arimoto algorithm, in both continuous and discrete-time, using **contraction mapping-based analysis and design**:

$$u_{k+1}(t) = u_k(t) + \Gamma(s)e_k(t)$$

- Various researchers have used **gradient methods** to optimize the gain G_k in:

$$u_{k+1}(t) = u_k(t) + G_k e_k(t+1)$$

- **Norm-optimal ILC** works in the state space. For the plant:

$$x_{k+1}(t+1) = Ax_{k+1} + Bu_{k+1}$$

find u_{k+1} to minimize

$$J = \|u_{k+1} - u_k\|^2 + |e_{k+1}\|^2$$

- It is also useful for analysis and design use **frequency domain-based learning control**, for example:

$$U_{k+1}(s) = L(s)[U_k(s) + aE_k(s)]$$

- A significant amount of work has also been done for nonlinear ILC, leading to the so-called **energy function approach**, using what are called control Lyapunov functions.
- Lots of work using **fuzzy-logic and artificial neural networks**.

Operator-Theoretic Convergence Condition

- **Theorem** (Moore, *et al.*, 1988): For the plant $y_k = T_s u_k$, the linear time-invariant learning control algorithm

$$u_{k+1} = T_u u_k + T_e(y_d - y_k)$$

converges to a fixed point $u^*(t)$ given by

$$u^*(t) = (I - T_u + T_e T_s)^{-1} T_e y_d(t)$$

with a final error

$$e^*(t) = \lim_{k\to\infty}(y_k - y_d) = (I - T_s(I - T_u + T_e T_s)^{-1} T_e) y_d(t)$$

defined on the interval (t_0, t_f) if $\| T_u - T_e T_s \|_i < 1$

Operator-Theoretic Convergence Condition

- **Theorem** (Moore, *et al.*, 1988): For the plant $y_k = T_s u_k$, the linear time-invariant learning control algorithm

$$u_{k+1} = T_u u_k + T_e(y_d - y_k)$$

converges to a fixed point $u^*(t)$ given by

$$u^*(t) = (I - T_u + T_e T_s)^{-1} T_e y_d(t)$$

with a final error

$$e^*(t) = \lim_{k\to\infty}(y_k - y_d) = (I - T_s(I - T_u + T_e T_s)^{-1} T_e) y_d(t)$$

defined on the interval (t_0, t_f) if $\| T_u - T_e T_s \|_i < 1$

- **Observation**:
  - If $T_u = I$ then $\|e^*(t)\| = 0$ for all $t \in [t_o, t_f]$.
  - Otherwise the error will be non-zero.

Operator-Theoretic Convergence Condition

- **Theorem** (Moore, *et al.*, 1988): For the plant $y_k = T_s u_k$, the linear time-invariant learning control algorithm

$$u_{k+1} = T_u u_k + T_e(y_d - y_k)$$

converges to a fixed point $u^*(t)$ given by

$$u^*(t) = (I - T_u + T_e T_s)^{-1} T_e y_d(t)$$

with a final error

$$e^*(t) = \lim_{k\to\infty}(y_k - y_d) = (I - T_s(I - T_u + T_e T_s)^{-1} T_e) y_d(t)$$

defined on the interval (t_0, t_f) if $\| T_u - T_e T_s \|_i < 1$

- **Observation**:
  - If $T_u = I$ then $\|e^*(t)\| = 0$ for all $t \in [t_o, t_f]$.
  - Otherwise the error will be non-zero.
- **Conclusion**: The essential effect of ILC is to *produce the output of the best possible inverse of the system in the direction of* y_d.

For Further Reading ...

- K. L. Moore, M. Dahleh, and S. P. Bhattacharyya, "Iterative learning control: a survey and new results," *J. of Robotic Systems*, 9(5):563–594, 1992.
- K.L. Moore, "Iterative learning control - an expository overview," *Applied & Computational Controls, Signal Processing, and Circuits*, 1(1):151–241, 1999.
- D.A. Bristow, M. Tharayil, and A. G. Alleyne, "A Survey of Iterative Learning Control," *IEEE Control Systems Magazine*, 26(3): 96–114, 2006.
- H. S. Ahn, Y. Q. Chen, and K. L. Moore, "Iterative learning control: brief survey and categorization 1998 – 2004," *IEEE Trans. on Systems, Man, and Cybernetics, Part–C*, Vol. 37, 2007.
- Y. Wang, F. Gao, F. J. Doyle III, "Survey on Iterative Learning Control, Repetitive Control, and Run-to-Run Control," *Journal of Process Contro*, Vol. 19, No. 10, pp. 1589-1600, 2009.
- Jian-Xin Xu, "A survey on iterative learning control for nonlinear systems," *International Journal of Control*, Vol. 84, No. 7, pp. 1275-1294, 2011.

ILC and Repetitive Control

- Much work has shown that RC can be viewed as a limiting case of ILC as the trial length goes to infinity
- For more see
  - "Unified Analysis of Iterative Learning and Repetitive Controllers in Trial Domain," Goele Pipeleers and Kevin L. Moore, *IEEE Transactions on Automatic Control*, Vol. 59, No. 4, pp. 953-965, April 2014.

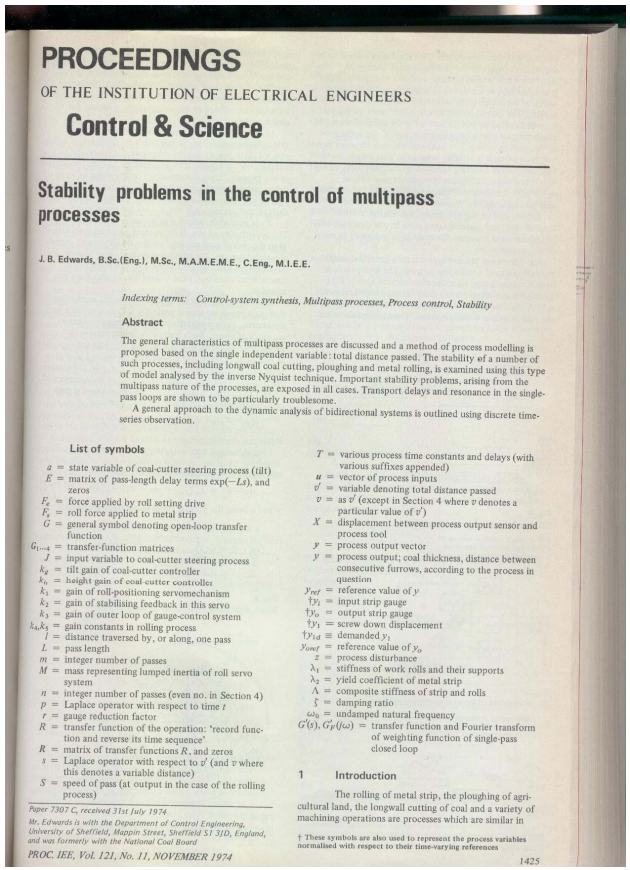
(Discrete) Repetitive Processes (DRP)

- There are a number of repetitive processes where the results of the previous iteration affect the plant seen by the controller in the next iteration
- Material removal or application processes are prototypical
- **Examples:**
  - Diffusion process
  - Digging
  - Multi-pass welding
  - Soil compaction

Rather than ILC, these processes better fit a class of systems called **Repetitive Processes**

Edwards, Proc. IEE (1974)

- J. B. Edwards, "Stability problems in the control of linear multipass processes," *Proc. IEE*, 121(11):1425–1431, 1974.



PROCEEDINGS

OF THE INSTITUTION OF ELECTRICAL ENGINEERS

Control & Science

Stability problems in the control of multipass processes

J. B. Edwards, B.Sc.(Eng.), M.Sc., M.A.M.E.M.E., C.Eng., M.I.E.E.

Indexing terms: Control-system synthesis, Multipass processes, Process control, Stability

Abstract

The general characteristics of multipass processes are discussed and a method of process modelling is proposed based on the single independent variable: total distance passed. The stability of a number of such processes, including longwall coal cutting, ploughing and metal rolling, is examined using this type of model analysed by the inverse Nyquist technique. Important stability problems, arising from the multipass nature of the processes, are exposed in all cases. Transport delays and resonance in the single-pass loops are shown to be particularly troublesome.

A general approach to the dynamic analysis of bidirectional systems is outlined using discrete time-series observation.

List of symbols

a = state variable of coal-cutter steering process (tilt)
E = matrix of pass-length delay terms $\exp(-Ls)$, and zeros
F_c = force applied by roll setting drive
F_r = roll force applied to metal strip
G = general symbol denoting open-loop transfer function
$G_{1\ldots4}$ = transfer-function matrices
J = input variable to coal-cutter steering process
k_a = tilt gain of coal-cutter controller
k_c = height gain of coal-cutter controller
k_1 = gain of roll-positioning servomechanism
k_2 = gain of stabilising feedback in this servo
k_3 = gain of outer loop of gauge-control system
k_4, k_5 = gain constants in rolling process
l = distance traversed by, or along, one pass
L = pass length
m = integer number of passes
M = mass representing lumped inertia of roll servo system
n = integer number of passes (even no. in Section 4)
p = Laplace operator with respect to time t
r = gauge reduction factor
R = transfer function of the operation: 'record function and reverse its time sequence'
R = matrix of transfer functions R, and zeros
s = Laplace operator with respect to v' (and v where this denotes a variable distance)
S = speed of pass (at output in the case of the rolling process)
T = various process time constants and delays (with various suffixes appended)
u = vector of process inputs
v' = variable denoting total distance passed
v = as v' (except in Section 4 where v denotes a particular value of v')
X = displacement between process output sensor and process tool
y = process output vector
y = process output; coal thickness, distance between consecutive furrows, according to the process in question
y_{ref} = reference value of y
$\dagger y_i$ = input strip gauge
$\dagger y_o$ = output strip gauge
$\dagger y_s$ = screw down displacement
$\dagger y_{id} \equiv$ demanded y_i
y_{oref} = reference value of y_o
z = process disturbance
λ_1 = stiffness of work rolls and their supports
λ_2 = yield coefficient of metal strip
Λ = composite stiffness of strip and rolls
ζ = damping ratio
ω_0 = undamped natural frequency
$G'(s), G'_F(j\omega)$ = transfer function and Fourier transform of weighting function of single-pass closed loop

Paper 7307 C, received 31st July 1974

Mr. Edwards is with the Department of Control Engineering, University of Sheffield, Mappin Street, Sheffield S1 3JD, England, and was formerly with the National Coal Board

1 Introduction

The rolling of metal strip, the ploughing of agricultural land, the longwall cutting of coal and a variety of machining operations are processes which are similar in

† These symbols are also used to represent the process variables normalised with respect to their time-varying references

Typical DRP processes

- Seminal work was in the area of long-wall coal-mining (Edwards, et al)
- Metal rolling
  - Both of these are described in detail in *Control Systems Theory and Applications for Linear Repetitive Processes*, Rogers, Galkowski, and Owens, Lecture notes in Control and Information Sciences, no. 349, Springer-Verlag, Berlin Heidelberg, 2007.
- Stroke rehab/adaptive physical therapy
- Additive manufacturing

Modeling Repetitive Processes

Normal Discrete-Time Linear Model (no repetition)

$$x(p+1) = Ax(p) + Bu(p)$$

$$y(p) = Cx(p) + Du(p) \qquad 0 \le p \le \alpha$$

Modeling Repetitive Processes

Normal Discrete-Time Linear Model (no repetition)

$$x(p+1) = Ax(p) + Bu(p)$$
$$y(p) = Cx(p) + Du(p) \qquad 0 \le p \le \alpha$$

p is time (or space)

Modeling Repetitive Processes

Normal Discrete-Time Linear Model (no repetition)

$$x(p+1) = Ax(p) + Bu(p)$$
$$y(p) = Cx(p) + Du(p) \qquad 0 \le p \le \alpha$$

p is time (or space)

Normal Discrete-Time Linear Model (with repetition)

$$x_k(p+1) = Ax_k(p) + Bu_k(p)$$
$$y_k(p) = Cx_k(p) + Du_k(p) \qquad 0 \le p \le \alpha$$

Modeling Repetitive Processes

Normal Discrete-Time Linear Model (no repetition)

$$x(p+1) = Ax(p) + Bu(p)$$
$$y(p) = Cx(p) + Du(p) \qquad 0 \le p \le \alpha$$

p is time (or space)

Normal Discrete-Time Linear Model (with repetition)

$$x_k(p+1) = Ax_k(p) + Bu_k(p)$$
$$y_k(p) = Cx_k(p) + Du_k(p) \qquad 0 \le p \le \alpha$$

k is trial or iteration

Iterative Learning Control

Normal Discrete-Time ILC

$$x_k(p+1) = Ax_k(p) + Bu_k(p)$$
$$y_k(p) = Cx_k(p) + Du_k(p) \qquad 0 \le p \le \alpha$$

$$u_{k+1}(p) = u_k(p) + \Gamma(y_d(p+1) - y_k(p+1))$$

Iterative Learning Control

Normal Discrete-Time ILC

$$x_k(p+1) = Ax_k(p) + Bu_k(p)$$
$$y_k(p) = Cx_k(p) + Du_k(p) \qquad 0 \le p \le \alpha$$

$$u_{k+1}(p) = u_k(p) + \Gamma(y_d(p+1) - y_k(p+1))$$

ILC adds this control law to normal linear model

- **Modifies control not plant**
- **Adds memory to the control law**

Discrete Repetitive Processes

Discrete Repetitive Processes (DRP)

$$x_{k+1}(p+1) = Ax_{k+1}(p) + Bu_{k+1}(p) + B_0 y_k(p)$$

$$y_{k+1}(p) = Cx_{k+1}(p) + Du_{k+1}(p) + D_0 y_k(p) \qquad 0 \le p \le \alpha$$

Boundary conditions :

$$x_{k+1}(0) = d_{k+1}$$

Discrete Repetitive Processes

Discrete Repetitive Processes (DRP)

$$x_{k+1}(p+1) = Ax_{k+1}(p) + Bu_{k+1}(p) + B_0 y_k(p)$$

$$y_{k+1}(p) = Cx_{k+1}(p) + Du_{k+1}(p) + D_0 y_k(p) \qquad 0 \leq$$

Boundary conditions :

$$x_{k+1}(0) = d_{k+1}$$

DRP adds this to normal linear model

- **Modifies plant, not control**
- **Adds memory to the plant**

Discrete Repetitive Processes

Discrete Repetitive Processes (DRP)

$$x_{k+1}(p+1) = Ax_{k+1}(p) + Bu_{k+1}(p) + B_0 y_k(p)$$

$$y_{k+1}(p) = Cx_{k+1}(p) + Du_{k+1}(p) + D_0 y_k(p) \qquad 0 \le p \le \alpha$$

Boundary conditions :

$$x_{k+1}(0) = d_{k+1}$$

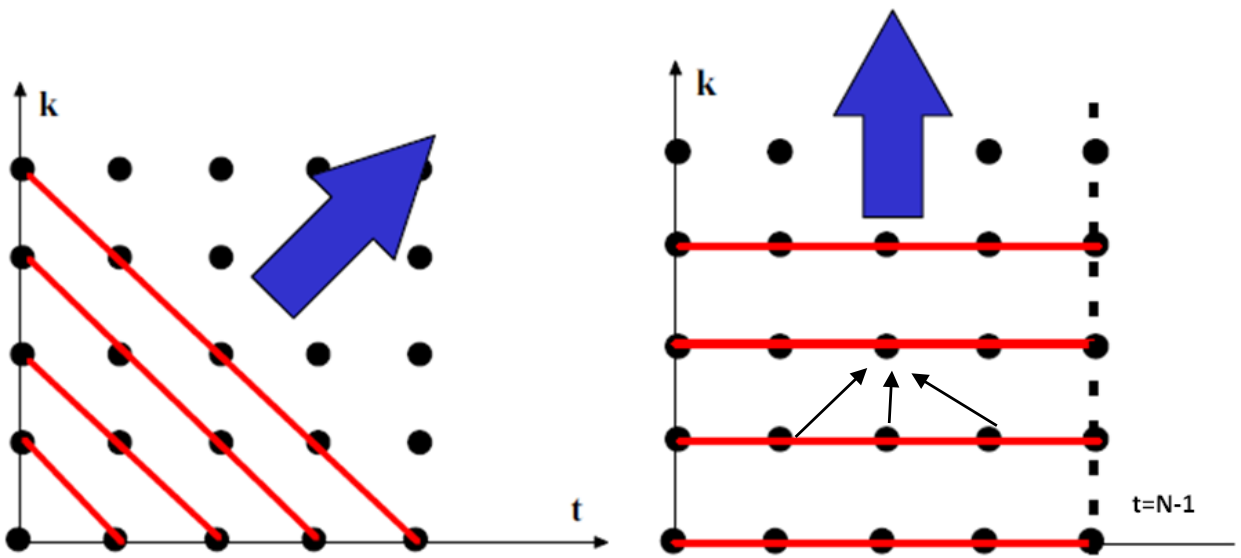
Control :

$$u_{k+1}(p) = f(y_d(p) - y_{k+1}(p))$$

Can then add "ILC-like" control

Repetitive Processes are 2-D Systems

- Repetitive processes propagate information in two independent directions where the duration of one of these is finite
- Repetitive Processes are a special type of 2-D systems (image concept due to Eric Rogers)



In This Two-Part Talk ...

- Part 0: Brief Overview of two of these paradigms:
  - ILC
  - DRP
- **Part 1: Analysis and design framework for such systems**
  - **Lifted models**
    - **The "supervector" framework of ILC**
    - **The update law using supervector notation**
    - **The ILC design problem**
    - **A lifting approach to DRPs**
  - **The *w*-transform and operator perspective**
  - **Generalization and closed-loop control of DRPs**
  - **Complete framework**
- Part 2: Design for discrete ILC/DRP systems

The "Supervector" Framework of ILC -1

- Consider an SISO, LTI discrete-time plant with relative degree m

$$\begin{aligned} Y(z) &= H(z)U(z) \\ &= (h_m z^{-m} + h_{m+1} z^{-(m+1)} + h_{m+2} z^{-(m+2)} + \cdots)U(z) \end{aligned}$$

The "Supervector" Framework of ILC -1

- Consider an SISO, LTI discrete-time plant with relative degree m

$$\begin{aligned} Y(z) &= H(z)U(z) \\ &= (h_m z^{-m} + h_{m+1} z^{-(m+1)} + h_{m+2} z^{-(m+2)} + \cdots) U(z) \end{aligned}$$

- By "lifting" along the time axis, for each trial k define

$$\begin{aligned} U_k &= [u_k(0), u_k(1), \cdots, u_k(N-1)]^T \\ Y_k &= [y_k(m), y_k(m+1), \cdots, y_k(m+N-1)]^T \\ Y_d &= [y_d(m), y_d(m+2), \cdots, y_d(m+N-1)]^T \end{aligned}$$

The "Supervector" Framework of ILC -2

- Thus the linear plant can be described by $Y_k = H_p U_k$ where (for m=1)

$$H_p = \begin{bmatrix} h_1 & 0 & 0 & \dots & 0 \\ h_2 & h_1 & 0 & \dots & 0 \\ h_3 & h_2 & h_1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_N & h_{N-1} & h_{N-2} & \dots & h_1 \end{bmatrix}$$

The "Supervector" Framework of ILC -2

- Thus the linear plant can be described by $Y_k = H_p U_k$ where (for m=1)

$$H_p = \begin{bmatrix} h_1 & 0 & 0 & \dots & 0 \\ h_2 & h_1 & 0 & \dots & 0 \\ h_3 & h_2 & h_1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_N & h_{N-1} & h_{N-2} & \dots & h_1 \end{bmatrix}$$

- The lower triangular matrix H_p is formed using the system's Markov parameters

The "Supervector" Framework of ILC -2

- Thus the linear plant can be described by $Y_k = H_p U_k$ where (for m=1)

$$H_p = \begin{bmatrix} h_1 & 0 & 0 & \dots & 0 \\ h_2 & h_1 & 0 & \dots & 0 \\ h_3 & h_2 & h_1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_N & h_{N-1} & h_{N-2} & \dots & h_1 \end{bmatrix}$$

- The lower triangular matrix H_p is formed using the system's Markov parameters
- Notice the non-causal shift ahead in forming the vectors U_k and Y_k

The "Supervector" Framework of ILC -3

- The lifting operation over a finite interval allows us to represent our dynamical system in R^1 as a static system in R^N

The "Supervector" Framework of ILC -3

- The lifting operation over a finite interval allows us to represent our dynamical system in R^1 as a static system in R^N

- We can extend this to
  - Time-varying systems
  - The ILC update law
  - Discrete repetitive processes (DRP)
  - Repetitive control (RC)
  - Affine nonlinear systems

The "Supervector" Framework of ILC -4

- For the linear, time-varying case, suppose we have the plant given by

$$\begin{aligned} x_k(t+1) &= A(t)x_k(t) + B(t)u_k(t) \\ y_k(t) &= C(t)x_k(t) + D(t)u_k(t) \end{aligned}$$

The "Supervector" Framework of ILC -4

- For the linear, time-varying case, suppose we have the plant given by

$$\begin{aligned} x_k(t+1) &= A(t)x_k(t) + B(t)u_k(t) \\ y_k(t) &= C(t)x_k(t) + D(t)u_k(t) \end{aligned}$$

- Then the same notation again results in $Y_k = H_p U_k$, where

The "Supervector" Framework of ILC -4

- For the linear, time-varying case, suppose we have the plant given by

$$\begin{aligned} x_k(t+1) &= A(t)x_k(t) + B(t)u_k(t) \\ y_k(t) &= C(t)x_k(t) + D(t)u_k(t) \end{aligned}$$

- Then the same notation again results in $Y_k = H_p U_k$, where

$$H_p = \begin{bmatrix} h_{1,0} & 0 & 0 & \dots & 0 \\ h_{2,0} & h_{1,1} & 0 & \dots & 0 \\ h_{3,0} & h_{2,1} & h_{1,2} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ h_{N,0} & h_{N-1,1} & h_{N-2,2} & \dots & h_{1,N-1} \end{bmatrix}$$

The Update Law Using Supervector Notation -1

- Suppose we have a simple "Arimoto-style" ILC update equation with a constant gain γ, described in our R^1 representation as

$$u_{k+1}(t) = u_k(t) + \gamma e_k(t+1)$$

The Update Law Using Supervector Notation -1

- Suppose we have a simple "Arimoto-style" ILC update equation with a constant gain γ, described in our R^1 representation as

$$u_{k+1}(t) = u_k(t) + \gamma e_k(t+1)$$

- In our R^N representation this becomes $U_{k+1} = U_k + \Gamma E_k$, where

The Update Law Using Supervector Notation -1

- Suppose we have a simple "Arimoto-style" ILC update equation with a constant gain γ, described in our R^1 representation as

$$u_{k+1}(t) = u_k(t) + \gamma e_k(t+1)$$

- In our R^N representation this becomes $U_{k+1} = U_k + \Gamma E_k$, where

$$\Gamma = \begin{bmatrix} \gamma & 0 & 0 & \dots & 0 \\ 0 & \gamma & 0 & \dots & 0 \\ 0 & 0 & \gamma & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & \gamma \end{bmatrix}$$

The Update Law Using Supervector Notation -2

- Suppose we filter with a causal LTI filter of relative degree m during the ILC update, written in our R^1 representation as

$$u_{k+1}(t) = u_k(t) + L(z)e_k(t+1)$$

The Update Law Using Supervector Notation -2

- Suppose we filter with a causal LTI filter of relative degree m during the ILC update, written in our R^1 representation as

$$u_{k+1}(t) = u_k(t) + L(z)e_k(t+1)$$

- In our R^N representation this becomes $U_{k+1} = U_k + LE_k$, where

The Update Law Using Supervector Notation -2

- Suppose we filter with a causal LTI filter of relative degree m during the ILC update, written in our R^1 representation as

$$u_{k+1}(t) = u_k(t) + L(z)e_k(t+1)$$

- In our R^N representation this becomes $U_{k+1} = U_k + LE_k$, where

$$L = \begin{bmatrix} L_m & 0 & 0 & \dots & 0 \\ L_{m+1} & L_m & 0 & \dots & 0 \\ L_{m+2} & L_{m+1} & L_m & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ L_{m+N-1} & L_{m+N-2} & L_{m+N-3} & \dots & L_m \end{bmatrix}$$

The Update Law Using Supervector Notation -3

- The supervector notation can also be applied to other ILC update schemes

The Update Law Using Supervector Notation -3

- The supervector notation can also be applied to other ILC update schemes
  - E.g., the Q-filter often introduced for stability (along the iteration domain) has the R^1 representation

$$u_{k+1}(t) = Q(z)(u_k(t) + L(z)e_k(t+1))$$

The Update Law Using Supervector Notation -3

- The supervector notation can also be applied to other ILC update schemes
  - E.g., the Q-filter often introduced for stability (along the iteration domain) has the R^1 representation

$$u_{k+1}(t) = Q(z)(u_k(t) + L(z)e_k(t+1))$$

  - The equivalent R^N representation is

$$U_{k+1} = Q(U_k + LE_k)$$

    where Q is a Toeplitz matrix formed using the Markov parameters of the filter $Q(z)$

The Update Law Using Supervector Notation -3

- The supervector notation can also be applied to other ILC update schemes
  - E.g., the Q-filter often introduced for stability (along the iteration domain) has the R^1 representation

$$u_{k+1}(t) = Q(z)(u_k(t) + L(z)e_k(t+1))$$

  - The equivalent R^N representation is

$$U_{k+1} = Q(U_k + LE_k)$$

    where Q is a Toeplitz matrix formed using the Markov parameters of the filter $Q(z)$
- We may also consider time-varying and noncausal filters in the ILC update law

The Update Law Using Supervector Notation -4

- A causal (in time), time-varying filter in the ILC update law might look like, for example

$$L = \begin{bmatrix} n_{1,0} & 0 & 0 & \dots & 0 \\ n_{2,0} & n_{1,1} & 0 & \dots & 0 \\ n_{3,0} & n_{2,1} & n_{1,2} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ n_{N,0} & n_{N-1,1} & n_{N-2,2} & \dots & n_{1,N-1} \end{bmatrix}$$

The Update Law Using Supervector Notation -5

- A non-causal (in time), time-invariant averaging filter in the ILC update law might look like, for example

$$L = \begin{bmatrix} K & K & 0 & 0 & \cdots & 0 & 0 & 0 \\ 0 & K & K & 0 & \cdots & 0 & 0 & 0 \\ 0 & 0 & K & K & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & K & K & 0 \\ 0 & 0 & 0 & 0 & \cdots & 0 & K & K \\ 0 & 0 & 0 & 0 & \cdots & 0 & 0 & K \end{bmatrix}$$

The ILC Design Problem -1

- The design of an ILC controller can be thought of as the selection of the matrix L

The ILC Design Problem -1

- The design of an ILC controller can be thought of as the selection of the matrix L
  - For a causal ILC updating law, L will be in lower-triangular Toeplitz form

The ILC Design Problem -1

- The design of an ILC controller can be thought of as the selection of the matrix L
  - For a causal ILC updating law, L will be in lower-triangular Toeplitz form
  - For a noncausal ILC updating law, L will be in upper-triangular Toeplitz form

The ILC Design Problem -1

- The design of an ILC controller can be thought of as the selection of the matrix L
  - For a causal ILC updating law, L will be in lower-triangular Toeplitz form
  - For a noncausal ILC updating law, L will be in upper-triangular Toeplitz form
  - For the popular zero-phase learning filter, L will be in a symmetrical band diagonal form

The ILC Design Problem -1

- The design of an ILC controller can be thought of as the selection of the matrix L
  - For a causal ILC updating law, L will be in lower-triangular Toeplitz form
  - For a noncausal ILC updating law, L will be in upper-triangular Toeplitz form
  - For the popular zero-phase learning filter, L will be in a symmetrical band diagonal form
  - L can also be fully populated

The ILC Design Problem -2

- Motivated by these comments, we will refer to the "causal" and "non-causal" elements of a general matrix Γ as follows

$$\Gamma = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} & \cdots & \gamma_{1N} \\ \gamma_{21} & \gamma_{22} & \gamma_{23} & \text{noncausal} & \gamma_{2N} \\ \gamma_{31} & \gamma_{32} & \gamma_{33} & \cdots & \gamma_{3N} \\ \vdots & \text{causal} & \vdots & \ddots & \vdots \\ \gamma_{N1} & \gamma_{N2} & \gamma_{N3} & \cdots & \gamma_{NN} \end{bmatrix}$$

The ILC Design Problem -2

- Motivated by these comments, we will refer to the "causal" and "non-causal" elements of a general matrix Γ as follows

$$\Gamma = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} & \cdots & \gamma_{1N} \\ \gamma_{21} & \gamma_{22} & \gamma_{23} & \text{noncausal} & \gamma_{2N} \\ \gamma_{31} & \gamma_{32} & \gamma_{33} & \cdots & \gamma_{3N} \\ \vdots & \text{causal} & \vdots & \ddots & \vdots \\ \gamma_{N1} & \gamma_{N2} & \gamma_{N3} & \cdots & \gamma_{NN} \end{bmatrix}$$

The diagonal elements of Γ are referred to as "Arimoto" gains

A Lifting Approach to DRPs -1

- Consider a SISO, LTI, DRP defined on $0 \leq t \leq N$ and $k \in [0, \infty)$

$$\begin{aligned} x_{k+1}(t+1) &= Ax_{k+1}(t) + Bu_{k+1}(t) + B_0 y_k(t) \\ y_{k+1}(t) &= Cx_{k+1}(t) + Du_{k+1}(t) + D_0 y_k(t) \end{aligned}$$

A Lifting Approach to DRPs -1

- Consider a SISO, LTI, DRP defined on $0 \leq t \leq N$ and $k \in [0, \infty)$

$$\begin{aligned} x_{k+1}(t+1) &= Ax_{k+1}(t) + Bu_{k+1}(t) + B_0 y_k(t) \\ y_{k+1}(t) &= Cx_{k+1}(t) + Du_{k+1}(t) + D_0 y_k(t) \end{aligned}$$

- We describe this process in the complex frequency domain (using usual z-transform notation) as

$$Y_{k+1} = H^u(z)U_{k+1}(z) + H^y(z)Y_k(z)$$

A Lifting Approach to DRPs -1

- Consider a SISO, LTI, DRP defined on $0 \leq t \leq N$ and $k \in [0, \infty)$

$$\begin{aligned} x_{k+1}(t+1) &= Ax_{k+1}(t) + Bu_{k+1}(t) + B_0 y_k(t) \\ y_{k+1}(t) &= Cx_{k+1}(t) + Du_{k+1}(t) + D_0 y_k(t) \end{aligned}$$

- We describe this process in the complex frequency domain (using usual z-transform notation) as

$$Y_{k+1} = H^u(z)U_{k+1}(z) + H^y(z)Y_k(z)$$

where

$$\begin{aligned} H^u(z) &= C(zI - A)^{-1}B + D \\ H^y(z) &= C(zI - A)^{-1}B_0 + D_0 \end{aligned}$$

A Lifting Approach to DRPs -2

- We convert the 2-D SISO DRP into a 1-D MIMO system by using "supervectors"

$$\begin{aligned} U_k &= (u_k(0), u_k(1), \cdots, u_k(N-1))^T \\ Y_k &= (y_k(0), y_k(1), \cdots, y_k(N-1))^T \end{aligned}$$

A Lifting Approach to DRPs -2

- We convert the 2-D SISO DRP into a 1-D MIMO system by using "supervectors"

$$\begin{aligned} U_k &= (u_k(0), u_k(1), \cdots, u_k(N-1))^T \\ Y_k &= (y_k(0), y_k(1), \cdots, y_k(N-1))^T \end{aligned}$$

- This gives the lifted representation

$$Y_{k+1} = H^u U_{k+1} + H^y Y_k$$

where H^u and H^y are lower-triangular Toeplitz matrices of rank N whose elements are the Markov parameters of the system

A Lifting Approach to DRPs -3

$$H^u = \begin{bmatrix} h_0^u & 0 & \cdots & 0 \\ h_1^u & h_0^u & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ h_{N-1}^u & h_{N-2}^u & \cdots & h_0^u \end{bmatrix}$$

$$H^y = \begin{bmatrix} h_0^y & 0 & \cdots & 0 \\ h_1^y & h_0^y & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ h_{N-1}^y & h_{N-2}^y & \cdots & h_0^y \end{bmatrix}$$

$h_0^u = D, h_0^y = D_0$ and for $i \geq 1$, $h_i^u = CA^{i-1}B, h_i^y = CA^{i-1}B_0$

Complex (Iteration) Variable Representation

- Let w be a complex variable and for each $t \in [0, N]$, let the (one-sided) w-transform $W(\cdot)$ be defined as

$$W(\{u_k(t)\}) \doteq \sum_{k=0}^{\infty} u_k(t) w^{-k}$$

Complex (Iteration) Variable Representation

- Let w be a complex variable and for each $t \in [0, N]$, let the (one-sided) w-transform $W(\cdot)$ be defined as

$$W(\{u_k(t)\}) \doteq \sum_{k=0}^{\infty} u_k(t) w^{-k}$$

- Two useful properties of the w-transform are
  1. *Shift Property*: Assuming $u_j = 0$ for all $j < 0$

$$W(\{u_{k-n}(t)\}) = w^{-n} W(\{u_k(t)\})$$

Complex (Iteration) Variable Representation

- Let w be a complex variable and for each $t \in [0, N]$, let the (one-sided) w-transform $W(\cdot)$ be defined as

$$W(\{u_k(t)\}) \doteq \sum_{k=0}^{\infty} u_k(t) w^{-k}$$

- Two useful properties of the w-transform are
  1. *Shift Property*: Assuming $u_j = 0$ for all $j < 0$

$$W(\{u_{k-n}(t)\}) = w^{-n} W(\{u_k(t)\})$$

  2. *Final Value Theorem*:

$$\lim_{k \to \infty} X_k = \lim_{w \to 1} (w-1) X(w)$$

  when the limit exists (e.g., if X_k is "stable")

Complex (Iteration) Variable Representation

- Let w be a complex variable and for each $t \in [0, N]$, let the (one-sided) w-transform $W(\cdot)$ be defined as

$$W(\{u_k(t)\}) \doteq \sum_{k=0}^{\infty} u_k(t) w^{-k}$$

- Two useful properties of the w-transform are
  1. *Shift Property*: Assuming $u_j = 0$ for all $j < 0$

$$W(\{u_{k-n}(t)\}) = w^{-n} W(\{u_k(t)\})$$

  2. *Final Value Theorem*:

$$\lim_{k \to \infty} X_k = \lim_{w \to 1} (w-1) X(w)$$

  when the limit exists (e.g., if X_k is "stable")
- w-transform is the "z-operator" in the iteration domain.

Operator Perspective of Basic ILC Problem

- For our "plant," $Y_k = HU_k$, applying the w-transform results in $Y(w) = HU(w)$, a MIMO, constant plant, where $U(w)$ and $Y(w)$ are the w-transforms of U_k and Y_k, respectively

Operator Perspective of Basic ILC Problem

- For our "plant," $Y_k = HU_k$, applying the w-transform results in $Y(w) = HU(w)$, a MIMO, constant plant, where $U(w)$ and $Y(w)$ are the w-transforms of U_k and Y_k, respectively
- Apply w-transform to the ILC update algorithm $U_{k+1} = U_k + LE_k$ to get:

$$wU(w) = U(w) + LE(w)$$

Operator Perspective of Basic ILC Problem

- For our "plant," $Y_k = HU_k$, applying the w-transform results in $Y(w) = HU(w)$, a MIMO, constant plant, where $U(w)$ and $Y(w)$ are the w-transforms of U_k and Y_k, respectively
- Apply w-transform to the ILC update algorithm $U_{k+1} = U_k + LE_k$ to get:

$$wU(w) = U(w) + LE(w)$$

- This can be written as

$$E(w) = C(w)U(w)$$

where

$$C(w) = \frac{1}{(w-1)}L$$

ILC as a MIMO Control System

- The term

$$C(w) = \frac{1}{(w-1)}L$$

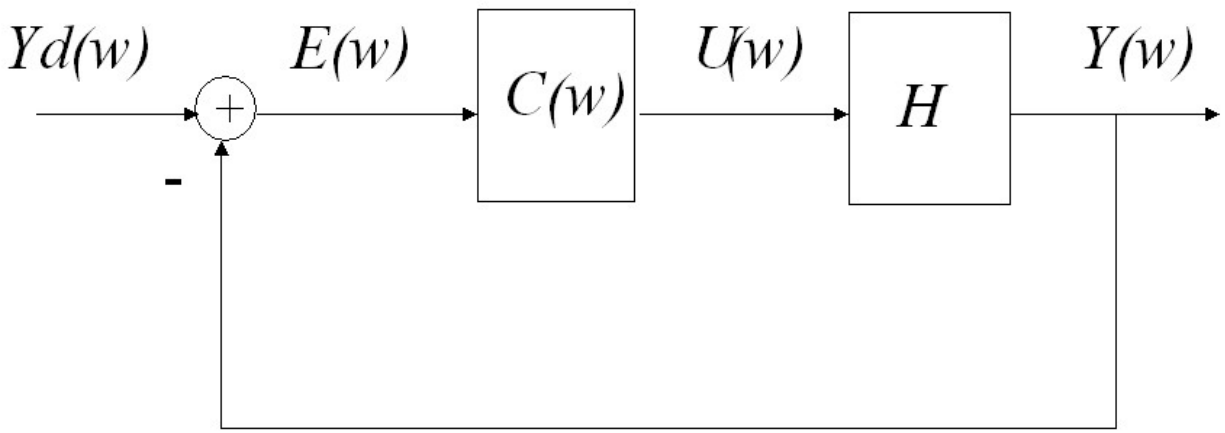
is effectively the controller of the system (in the repetition domain). This can be depicted as:

ILC as a MIMO Control System

- The term

$$C(w) = \frac{1}{(w-1)}L$$

is effectively the controller of the system (in the repetition domain). This can be depicted as:



Operator Perspective of DRP Problem

- Applying the w-transform to our lifted vectors and applying the shift property gives

$$W(\{Y_{k+1}\}) \;=\; W(\{H^u U_{k+1} + H^y Y_k\})$$

Operator Perspective of DRP Problem

- Applying the w-transform to our lifted vectors and applying the shift property gives

$$W(\{Y_{k+1}\}) = W(\{H^u U_{k+1} + H^y Y_k\})$$

$$wY(w) = wH^u U(w) + H^y Y(w)$$

Operator Perspective of DRP Problem

- Applying the w-transform to our lifted vectors and applying the shift property gives

$$W(\{Y_{k+1}\}) = W(\{H^u U_{k+1} + H^y Y_k\})$$

$$wY(w) = wH^u U(w) + H^y Y(w)$$

or

$$\begin{aligned} Y(w) &= w(wI - H^y)^{-1} H^u U(w) \\ &= wH^u (wI - H^y)^{-1} U(w) \end{aligned}$$

Operator Perspective of DRP Problem

- Applying the w-transform to our lifted vectors and applying the shift property gives

$$W(\{Y_{k+1}\}) = W(\{H^u U_{k+1} + H^y Y_k\})$$

$$wY(w) = wH^u U(w) + H^y Y(w)$$

or

$$\begin{aligned} Y(w) &= w(wI - H^y)^{-1} H^u U(w) \\ &= wH^u (wI - H^y)^{-1} U(w) \end{aligned}$$

- Compare this to ILC: $Y(w) = HU(w)$ (thus $H = H_u$).

Higher-Order ILC in the Iteration Domain

- We can use these ideas to develop more general expressions ILC algorithms
- For example, a "higher-order" ILC algorithm could have the form:

$$u_{k+1}(t) = k_1 u_k(t) + k_2 u_{k-1}(t) + \gamma e_k(t+1)$$

Higher-Order ILC in the Iteration Domain

- We can use these ideas to develop more general expressions ILC algorithms
- For example, a "higher-order" ILC algorithm could have the form:

$$u_{k+1}(t) = k_1 u_k(t) + k_2 u_{k-1}(t) + \gamma e_k(t+1)$$

which corresponds to:

$$C(w) = \frac{\gamma w}{w^2 - k_1 w - k_2}$$

Higher-Order ILC in the Iteration Domain

- We can use these ideas to develop more general expressions ILC algorithms
- For example, a "higher-order" ILC algorithm could have the form:

$$u_{k+1}(t) = k_1 u_k(t) + k_2 u_{k-1}(t) + \gamma e_k(t+1)$$

which corresponds to:

$$C(w) = \frac{\gamma w}{w^2 - k_1 w - k_2}$$

- Next we show how to extend these notions to develop an algebraic (matrix fraction) description of the ILC problem

Closed-Loop Control of DRPs -1

- Later we consider controller design in detail. Here we present the general linear case.

Closed-Loop Control of DRPs -1

- Later we consider controller design in detail. Here we present the general linear case.
- As in ILC, use filtered errors from m previous (and current) passes and filtered inputs from $n \geq m$ previous passes

Closed-Loop Control of DRPs -1

- Later we consider controller design in detail. Here we present the general linear case.
- As in ILC, use filtered errors from m previous (and current) passes and filtered inputs from $n \geq m$ previous passes

$$\begin{aligned} u_{k+1}(t) &= -D_{n-1}(z)u_k(t) - \cdots - D_0(z)u_{k-n+1}(t) \\ &+ N_m(z)e_{k+1}(t) + \cdots + N_0(z)e_{k-m+1}(t) \end{aligned}$$

Closed-Loop Control of DRPs -1

- Later we consider controller design in detail. Here we present the general linear case.
- As in ILC, use filtered errors from m previous (and current) passes and filtered inputs from $n \geq m$ previous passes

$$\begin{aligned} u_{k+1}(t) &= -D_{n-1}(z)u_k(t) - \cdots - D_0(z)u_{k-n+1}(t) \\ &+ N_m(z)e_{k+1}(t) + \cdots + N_0(z)e_{k-m+1}(t) \end{aligned}$$

where $D_i(z)$ and $N_i(z)$ denote discrete-time filters and the error is $e_k(t) = y_k^d(t) - y_k(t)$

Closed-Loop Control of DRPs -1

- Later we consider controller design in detail. Here we present the general linear case.
- As in ILC, use filtered errors from m previous (and current) passes and filtered inputs from $n \geq m$ previous passes

$$\begin{aligned} u_{k+1}(t) &= -D_{n-1}(z)u_k(t) - \cdots - D_0(z)u_{k-n+1}(t) \\ &+ N_m(z)e_{k+1}(t) + \cdots + N_0(z)e_{k-m+1}(t) \end{aligned}$$

  where $D_i(z)$ and $N_i(z)$ denote discrete-time filters and the error is $e_k(t) = y_k^d(t) - y_k(t)$
- Note (1) we allow iteration-varying references $y_k^d(t)$

Closed-Loop Control of DRPs -1

- Later we consider controller design in detail. Here we present the general linear case.
- As in ILC, use filtered errors from m previous (and current) passes and filtered inputs from $n \geq m$ previous passes

$$\begin{aligned} u_{k+1}(t) &= -D_{n-1}(z)u_k(t) - \cdots - D_0(z)u_{k-n+1}(t) \\ &+ N_m(z)e_{k+1}(t) + \cdots + N_0(z)e_{k-m+1}(t) \end{aligned}$$

  where $D_i(z)$ and $N_i(z)$ denote discrete-time filters and the error is $e_k(t) = y_k^d(t) - y_k(t)$
- Note (1) we allow iteration-varying references $y_k^d(t)$ and (2) all filters may be non-causal in time (except the current pass factor $N_m(z)$)

Closed-Loop Control of DRPs -2

- To proceed, perform lifting to get

$$\begin{aligned} U_{k+1} &= -D_{n-1}U_k - \cdots - D_0 U_{k-n+1} \\ &+ N_m E_{k+1} + N_{m-1}E_k + \cdots + N_0 E_{k-m+1} \end{aligned}$$

Closed-Loop Control of DRPs -2

- To proceed, perform lifting to get

$$\begin{aligned} U_{k+1} &= -D_{n-1}U_k - \cdots - D_0 U_{k-n+1} \\ &+ N_m E_{k+1} + N_{m-1}E_k + \cdots + N_0 E_{k-m+1} \end{aligned}$$

- Then apply the w-transform and collect terms to get

$$D_c(w)U(w) = N_c(w)E(w),$$

where

$$\begin{aligned} D_c(w) &= w^n I + D_{n-1}w^{n-1} + \cdots + D_1 w + D_0 \\ N_c(w) &= N_m w^m + N_{m-1}w^{m-1} + \cdots + N_1 w + N_0 \end{aligned}$$

Closed-Loop Control of DRPs -3

- We can then write the controller as the matrix fraction

$$U(w) = C(w)E(w)$$

where

$$C(w) = D_c^{-1}(w)N_c(w)$$

Closed-Loop Control of DRPs -3

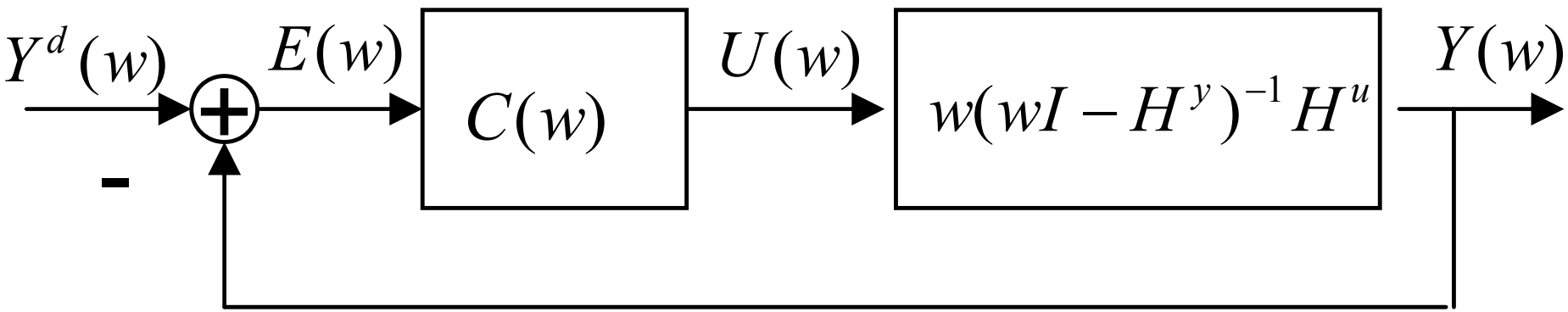
- We can then write the controller as the matrix fraction

$$U(w) = C(w)E(w)$$

where

$$C(w) = D_c^{-1}(w)N_c(w)$$

- The plant and controller combination can be shown as



Closed-Loop Control of DRPs -4

- The figure above defines a general MIMO control system, just as in discrete-time linear ILC

Closed-Loop Control of DRPs -4

- The figure above defines a general MIMO control system, just as in discrete-time linear ILC
- Thus we may consider standard MIMO analysis and design techniques

Closed-Loop Control of DRPs -4

- The figure above defines a general MIMO control system, just as in discrete-time linear ILC
- Thus we may consider standard MIMO analysis and design techniques
- There are two primary concerns: stability and performance, the latter often typified by a tracking requirement

Closed-Loop Control of DRPs -4

- The figure above defines a general MIMO control system, just as in discrete-time linear ILC
- Thus we may consider standard MIMO analysis and design techniques
- There are two primary concerns: stability and performance, the latter often typified by a tracking requirement
- For both ILC and DRP stability is equivalent to convergence along the iteration axis (as $k \to \infty$)

Closed-Loop Control of DRPs -4

- The figure above defines a general MIMO control system, just as in discrete-time linear ILC
- Thus we may consider standard MIMO analysis and design techniques
- There are two primary concerns: stability and performance, the latter often typified by a tracking requirement
- For both ILC and DRP stability is equivalent to convergence along the iteration axis (as $k \to \infty$)
- This problem reduces to making the roots of the pole polynomial $G_{cl}(w)$ lie in the open unit disk of the complex plane, where

$$G_{cl}(w) = wH^u[D_c(w)(wI - H^y) + N_c(w)wH^u]^{-1}N_c(w)$$

Complete Framework -1

- In ILC it is common to
  - Separate out current cycle control action, factoring the controller as

$$C(w) = \bar{C}_{ILC}(w) + C_{CITE}$$

where CITE stands for "Current Iteration"

Complete Framework -1

- In ILC it is common to
  - Separate out current cycle control action, factoring the controller as

$$C(w) = \bar{C}_{ILC}(w) + C_{CITE}$$

    where CITE stands for "Current Iteration"
  - Explicitly account for iteration-invariant references by using the internal model principle to further factor the ILC controller as

$$\bar{C}_{ILC}(w) = \frac{1}{w-1} C_{ILC}(w)$$

Complete Framework -1

- In ILC it is common to
  - Separate out current cycle control action, factoring the controller as

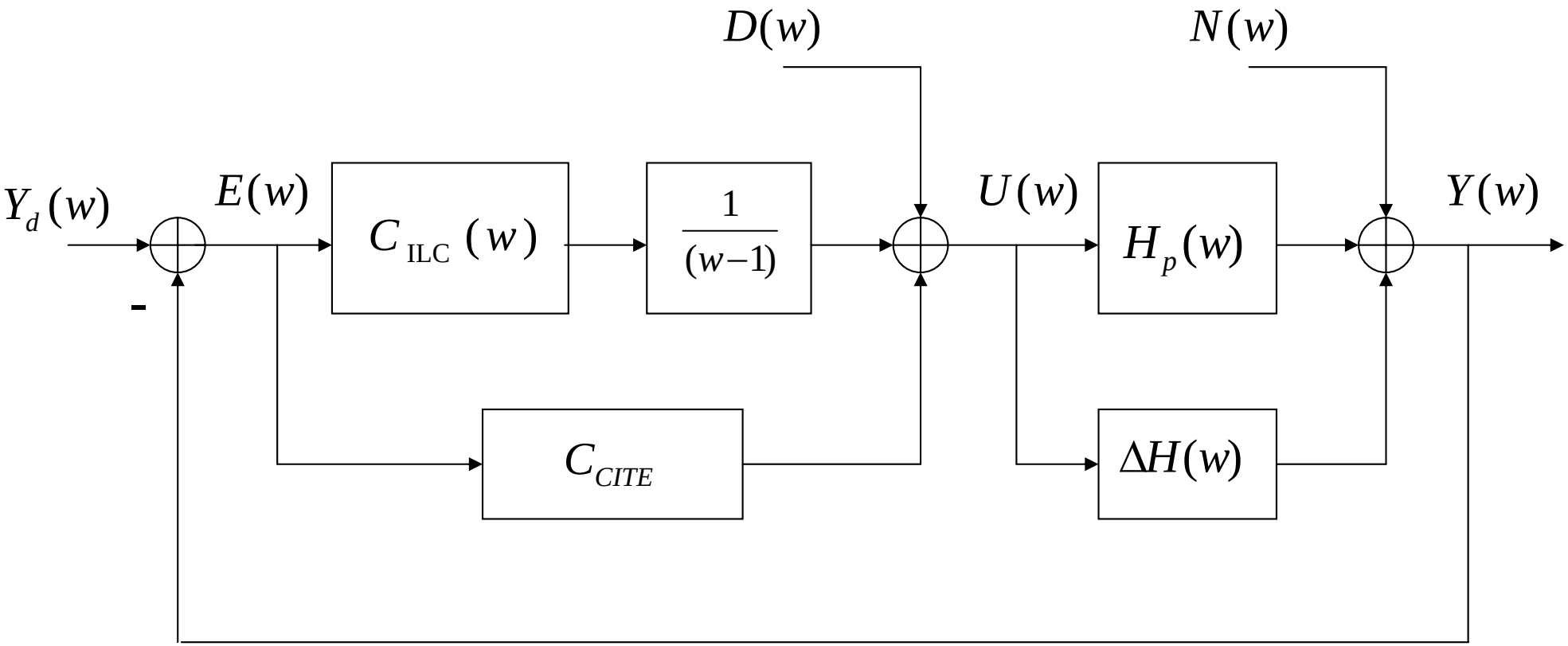
$$C(w) = \bar{C}_{ILC}(w) + C_{CITE}$$

    where CITE stands for "Current Iteration"
  - Explicitly account for iteration-invariant references by using the internal model principle to further factor the ILC controller as

$$\bar{C}_{ILC}(w) = \frac{1}{w-1} C_{ILC}(w)$$

  - Consider the possibility of noise, disturbances, and different types of uncertainty in the plant

Complete Framework -2



Categorization of Problems

Y_d	D	N	C	H	
$Y_d(z)$	0	0	$\Gamma(w-1)^{-1}$	H_p	Classical Arimoto algorithm
$Y_d(z)$	0	0	$\Gamma(w-1)^{-1}$	$H_p(w)$	Rogers'/Owens' multipass problem
$Y_d(z)$	$D(w)$	0	$C(w)$	H_p	General (higher-order) problem
$Y_d(w)$	$D(w)$	0	$C(w)$	H_p	General (higher-order) problem
$Y_d(w)$	$D(w)$	$N(w)$	$C(w)$	$H_p(w)+\Delta(w)$	**Most general problem**
$Y_d(z)$	$D(z)$	0	$C(w)$	$H(w)+\Delta(w)$	Frequency-domain uncertainty
$Y_d(z)$	0	$w(t), v(t)$	$\Gamma(w-1)^{-1}$	H_p	Least quadratic ILC
$Y_d(z)$	0	$w(t), v(t)$	$C(w)$	H_p	Stochastic ILC (general)
$Y_d(z)$	0	0	$\Gamma(w-1)^{-1}$	H^I	Interval ILC
$Y_d(z)$	$D(z)$	$w(t), v(t)$	$C(w)$	$H(z)+\Delta H(z)$	Time-domain H_∞ problem
$Y_d(z)$	$D(w)$	$w(k,t), v(k,t)$	$C(w)$	$H(w)+\Delta H(w)$	Iteration-domain H_∞ ILC
$Y_d(z)$	0	$w(k,t), v(k,t)$	$\Gamma(k)$	$H_p+\Delta H(k)$	Iteration-varying uncertainty and control
$Y_d(z)$	0	$\widetilde{H}$	Γ	H_p	Intermittent measurement problem
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Summary

- ILC, RC, and DRP are classes of **multi-pass systems** that can use data to improve their operation from pass-to-pass
- ILC is a special case of DRP

Summary

- ILC, RC, and DRP are classes of **multi-pass systems** that can use data to improve their operation from pass-to-pass
- ILC is a special case of DRP
- In the discrete-time case, **lifted models** provide a useful representation for analysis and design

Summary

- ILC, RC, and DRP are classes of **multi-pass systems** that can use data to improve their operation from pass-to-pass
- ILC is a special case of DRP
- In the discrete-time case, **lifted models** provide a useful representation for analysis and design
- By adopting an **operator transform** on the iteration domain (for fixed time), a general **MIMO framework** can be developed for control design problems

Summary

- ILC, RC, and DRP are classes of **multi-pass systems** that can use data to improve their operation from pass-to-pass
- ILC is a special case of DRP
- In the discrete-time case, **lifted models** provide a useful representation for analysis and design
- By adopting an **operator transform** on the iteration domain (for fixed time), a general **MIMO framework** can be developed for control design problems
- In **Part 2** we consider several **design** problems for these systems, using this framework

Further reading on the algebraic framework

- "A Matrix-Fraction Approach to Higher-Order Iterative Learning Control: 2-D Dynamics through Repetition-Domain Filtering," Kevin L. Moore, in *Proceedings of 2nd International Workshop on Multidimensional (nD) Systems*, pp. 99-104, Lower Selesia, Poland, June 2000

- "An Algebraic Approach to Iterative Learning Control," Jari Hatonen, David H. Owens, and Kevin L. Moore, *International Journal of Control*, vol. 77, no. 10, pp. 45-54, January 2004 (also in *Proceedings of the 17th IEEE International Symposium on Intelligent Control*, IEEE ISIC'02, Vancouver, British Columbia October 27-30, 2002)

- "Analysis of Linear Iterative Learning Scheme using Repetitive Process Theory," D. Owens, E. Rogers, and Kevin L. Moore, *Asian Journal of Control*, vol. 4, No. 1, Mar. 2002, pp. 90-98

- *Iterative Learning Control: An Optimization Paradigm*, David H. Owens, Advances in Industrial Control, Springer-Verlag, London, 2016.

- *Iterative Learning: Control Algorithms and Experimental Benchmarking*, Eric Rogers, Bing Chu, Christopher Freeman, and Paul Lewin, Wiley, 2023